How to Measure, Present, and Compare Parallel Performance

Lawrence A. Crowl
Oregon State University

Presentations of parallel performance can be easily and unintentionally distorted. Following some simple guidelines for measuring, presenting, and comparing performance can greatly improve your presentation's accuracy and effectiveness.

Programmers use parallelism to make their programs solve a single problem in less time or solve more (or larger) problems in a fixed time. Because performance is so important to parallel computing, people devote substantial effort to measuring, presenting, and comparing it. Unfortunately, many presentations are not very effective. They often require substantial interpretation, exaggerate or diminish performance differences, conceal important performance trends, or ignore significant computational costs. While this list of problems may seem like an account of fraud, most presentation problems may well be unintentional, arising from a lack of guidelines for crafting an effective presentation.

The guidelines in this article will help you clearly and effectively present parallel performance, and help you better interpret and critically evaluate other writers' presentations. When applying these guidelines, you should bear in mind three distinct goals:

- *Present information, not just data.* Simply publishing a list of all data points collected would communicate all the data available, but would communicate the information poorly.
- *Use space and time effectively.* On the printed page, two things limit your ability to communicate: the space available and the time the reader must spend to interpret your presentation. For only a few data points, simply listing them may minimize the space and time. For many data points, a carefully crafted graph or other visualization might work better.
- *Minimize distortion.* Many things can distort parallel performance, either intentionally or accidentally. Some distortion is inevitable, but choosing the right performance measures, presenting them well on paper, and making fair comparisons will reduce it.

1063-6552/94/ $4.00 © 1994 IEEE

Summary of guidelines

Make your presentation of parallel performance more effective by presenting information (not just data), by using space and time effectively, and by minimizing distortion:

- Measure elapsed time to account for CPU time *and* waiting time.
- Measure variance in elapsed time.
- Present elapsed time rather than speedup because speedup is a derivative measure, subject to different and sometimes undocumented derivations. Linear speed is visually similar to speedup, so you may use it instead.
- Use graphs whenever you have more than a few data points.
- Use linear speed plots (with zero origins) when absolute performance is important.
- Use log-log time plots when absolute performance is not important; their scale invariance lets you compare performance independently of base processor technology.
- Show measure points explicitly.
- Show variability in execution time when it is significant, preferably by plotting all data points.
- Limit the number of independent variables you change when making comparisons, and document those you do change. Document other independent variables needed to reproduce the experiment.
- Scale execution times only when comparing parallelizations, and rely on the scale invariance of log-log plots for all other comparisons.
- Show "perfect" performance as separate lines, rather than scaling the graph.

You might not be able to satisfy all three goals simultaneously; for example, tables of raw data generally minimize distortion, but require much time to interpret. You must choose a balance among information, space, time, and distortion. However, following these guidelines will often improve your presentation with respect to all three goals.

What to measure

Before presenting parallel performance, you must measure it. Many factors determine a parallel program's performance: the algorithm, the programming system, the machine, the problem, the execution time, and so on (see "What to compare"). You can directly control all of them except time. So, measure time.

Elapsed time

Measuring time seems pretty obvious. What is not so obvious is what kind of time to measure. Should you measure the CPU time of a single process within the program, the average CPU time of all processes, or the elapsed (wall clock) time?

- *Single-process CPU time.* Many researchers measure the time of a single process, under the assumption that it will be representative of all processes in the system. However, unless the operating system guarantees that all parallel processes receive exactly the same CPU time, some will use less than others. It may be impossible to determine which processes will use more or less CPU time, so the measured time may be random. If the measured process happens to consume less time, the measured computation will not accurately reflect the work done by the unmeasured processes. In Figure 1, for example, not all processes receive equal amounts of CPU time, and the CPU time of the measured processes (shown) may be significantly different from the average CPU time (not shown); the effect is particularly noticeable at 18 and 19 processors.
- *Average process CPU time.* You can ensure that you measure all the work done by measuring the system and user CPU time for all processes and then averaging. However, this approach does not account for the time spent waiting for locks, paging, or I/O, which can be a substantial fraction of the actual elapsed time for many parallel programs (see Figure 2). (In this and several other figures, the reference is to the source of the data in the graph.)
- *Elapsed (wall clock) time.* Measuring elapsed time accounts not only for the computational work, but also for any waiting for locks, paging, and I/O. Much of this waiting may be inherent in the program, and hence is an integral part of parallel performance.

While CPU time may be a tempting and convenient measure, the bottom line in parallel program performance is how long people must wait for a solution; therefore you should measure elapsed time.

But when measuring elapsed time, most writers prefer to show the program's performance on a dedicated machine; this avoids the reduced performance that results when the program must contend for machine resources with unrelated programs, such as on time-sharing systems. When it is not possible to measure on a dedicated machine, it is tempting to avoid using elapsed time because it is "unfair." However, avoiding elapsed time hides not only time spent waiting on the unrelated programs, but also the waiting time internal to the program. Instead, you should measure and present both average CPU time and elapsed time, and then describe the mix

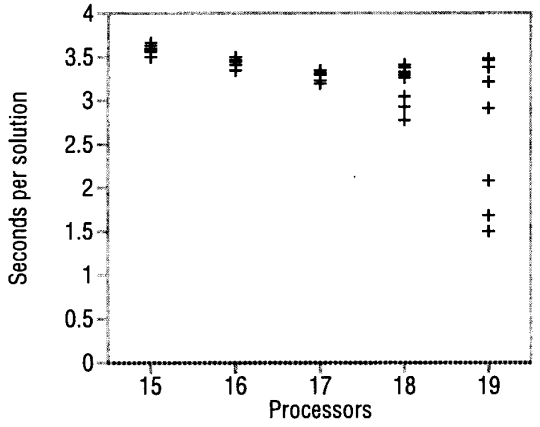


Figure 1. Variation when measuring single-process CPU time.

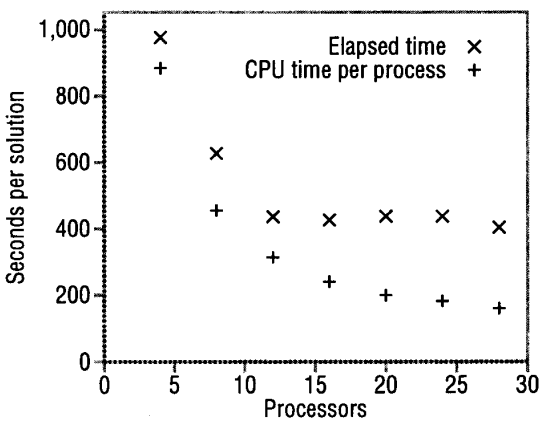


Figure 2. Elapsed time versus average CPU time per process.[1]

of unrelated programs contending for system resources. By being explicit about the contention, you let the reader determine if the job mix is significant or not.

Time variation

You may be tempted to measure your program's execution time only once. Resist the temptation. Parallel programs must often deal with random events in the system: memory accesses may be reordered, operating system daemons may or may not execute, and so on. Because of this, parallel program execution time is not deterministic; it varies around some mean execution time. To ensure that the measured execution time is not some statistical fluke, you must measure it over several program runs and determine its mean and variance. A high variance may indicate problems with the program, such as high contention for locks. Without statistical measures of the program's execution time, you do not know if the one time you measured is an unlikely event, and therefore a distortion of reality. So, measure variation in execution time.

Caches

When executing a parallel algorithm several times, particularly within a single program, the first few runs might be significantly slower than subsequent runs. This occurs because modern systems rely heavily on caches, such as for main memory, virtual memory, and files. This transient behavior distorts performance results when programs that normally run multiple times are measured only during the first run, or when programs that normally run only once are measured over multiple runs.

To avoid distortions due to caches, you should adopt one of two measurement strategies:

- *Measure stable caches.* For programs that run repeatedly and whose cache behavior is relatively independent of the input data, you should measure several successive runs, and measure (or record) execution time only after the caches have stabilized.
- *Measure transient caches.* If the program will yield the same answer each time it runs, running it more than once in actual use is pointless. In this case, accurately measuring multiple execution times requires that any caches be flushed before each run, so that you include the transient cache startup behavior in each run. In addition to the memory cache, you must worry about address translation caches, page migration and replication caches, file system caches, and so on.

Occasionally the time needed to collect transient cache timing data exceeds the time available to the experiment. In this case, you should report both the execution time of the first run and the execution time after the cache is stable. The initial time is more accurate but less reliable, while the steady-state time is less accurate but more reliable. Presenting both lets the readers evaluate the caches' significance to performance.

Because nearly all modern systems use caches extensively, you should document clearly the cache measurement strategy you use.

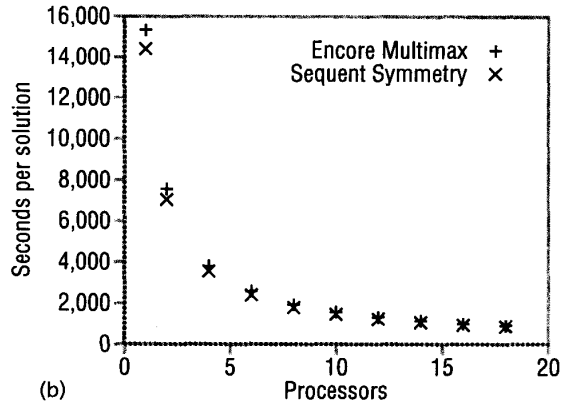
What to present

Once you have a statistical measure of the elapsed time, you must decide what information to present to readers. Simply present the elapsed time, and avoid presenting speedup where possible.

Speedup is a common way to present parallel performance because the results seem independent of the specific hardware. However, speedup has several problems because it is a derivative measure and there are many choices on how to derive it. You cannot assume your reader will infer the "right" speedup, because there is

Number of processors	Sequent Symmetry	Encore Multimax
1	14,417.9	15,338.0
2	7,058.0	7,566.1
4	3,566.2	3,807.2
6	2,412.8	2,584.5
8	1,799.4	1,934.9
10	1,471.9	1,577.9
12	1,227.4	1,320.5
14	1,069.9	1,137.9
16	944.5	999.1
18	872.4	917.8

(a)



(b)

Figure 3. Table versus graph presentation.[1]

no clear consensus among parallel programmers on how to derive speedup.[2] Each time you present speedup, you force your reader to ask several questions to determine how you derived it and how to interpret your results.

- *Kind of speedup.* Are you using normal speedup (problem size is fixed), scaled speedup (problem size grows with the number of processors),[3] memory-bound speedup (problem size grows until memory is limited),[4] or accuracy speedup (problem parameters are adjusted to increasing accuracy)?[5] For scaled speedup, does the problem size grow linearly with the number of processors, or does the resulting complexity grow linearly with the number of processors? If scaling with complexity, are you scaling with worst-case or average-case complexity? For some problems, the complexity is not even known, which makes scaling with complexity problematic.
- *Sequential time base.* What is the base time used to calculate speedup: the parallel program on one processor, an equivalent sequential program, or the best-known sequential program? Does your machine sacrifice sequential performance for parallel scalability? Many do. If so, the sequential time will be artificially high, yielding overestimates on the speedup. In some cases, the problem is too large to execute on a single processor, so you must guess at a base time, which is ripe for distortion.
- *Resources available.* The concept of speedup implies that only the number of processors changes. However, as you use more processors in any real machine, you will likely get some combination of a larger cache, more physical memory, and more contention from system daemons. Because of growing resources, some programs get superlinear speedup! This causes no end of argument.[6]

Speedup is highly subject to misinterpretation. On the other hand, if you present elapsed time, you avoid making derivative interpretations, you avoid making your reader check your derivation, and you avoid making claims about your base case. In short, presenting elapsed time does not distort your results.

How to present

How you present information on paper can significantly affect how well you communicate. Mapping information to paper involves many factors, and balancing them requires care and taste.

The amount of data strongly influences the best way to present it. If you have only a few data points, list them in a table. It takes little space and is easy to interpret. An example table appears in Figure 3a, though with more points than I recommend. For more than three or four data points, use either a bar chart or graph (see Figure 3b). Bar charts work best when the independent variable is not quantitative, such as the type of machine or application. Graphs work best when the independent variable is quantitative, such as the number of processors. For example, the graph in Figure 3b presents information more effectively than the corresponding table. Your audience rarely needs to see the raw data; reserve it for an appendix, or better yet, anonymous FTP.

I will concentrate here on graphs; most of what I have to say about them applies equally well to bar charts. In addition, I will concentrate on the number of processors, p, as the independent variable and execution time, $t(p)$, as the dependent variable. There are other equally valid independent variables, such as system cost. Dependent variables may also differ depending on the application (profits, for example, or response time).

Plot functions

When designing a graph, you must choose a function that plots data onto paper. The simplest is a linear function. For example, each unit in the independent vari-

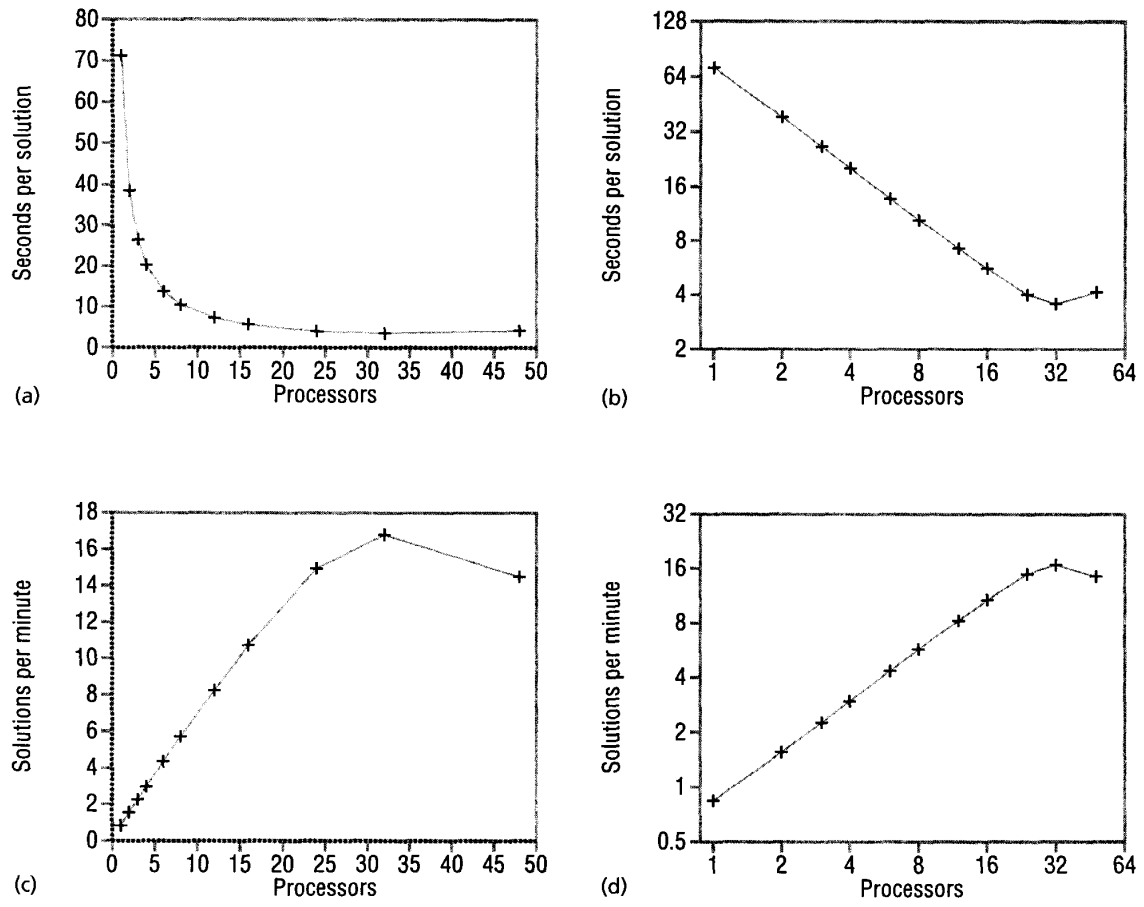


Figure 4. Showing qualitative changes: (a) linear time, (b) log-log time, (c) linear speed, (d) log-log speed.[7]

able might correspond to one centimeter horizontally on paper, and each unit in the dependent variable might correspond to two centimeters vertically on paper. I cannot know how large your units are, or how much paper you have, so I will concentrate on the general properties of four function types, not on their specific coefficients:

- *Linear time*: width proportional to the number of processors, and height proportional to time.
- *Linear speed*: width proportional to the number of processors, and height proportional to the inverse of time.
- *Log-log time*: width proportional to the logarithm of the number of processors, and height proportional to the logarithm of time.
- *Log-log speed*: width proportional to the logarithm of the number of processors, and height proportional to the logarithm of the inverse of time.

The ideal plot function would create graphs in which qualitative changes are obvious. Linear, absolute, and relative performance are linear on paper, and performance is scale-invariant. None of these functions satisfy all of these properties, but some are clearly better than others. Log-linear and linear-log plots are also possible, but they do not satisfy these properties well.

Qualitative changes should be obvious

A plot function must make qualitative changes easy to see. For example, Figure 4 shows the same parallel performance under each of the four plot functions. In looking for qualitative changes, we should be able to see where performance improves, levels off, and gets worse. These changes are easy to see in the linear speed, log-log time, and log-log speed plots, but difficult to see in the linear time plot. The linear time plot does not show qualitative changes well because performance at high numbers of processors receives very little visual space. Linear time plots show qualitative changes poorly, so you should avoid them.

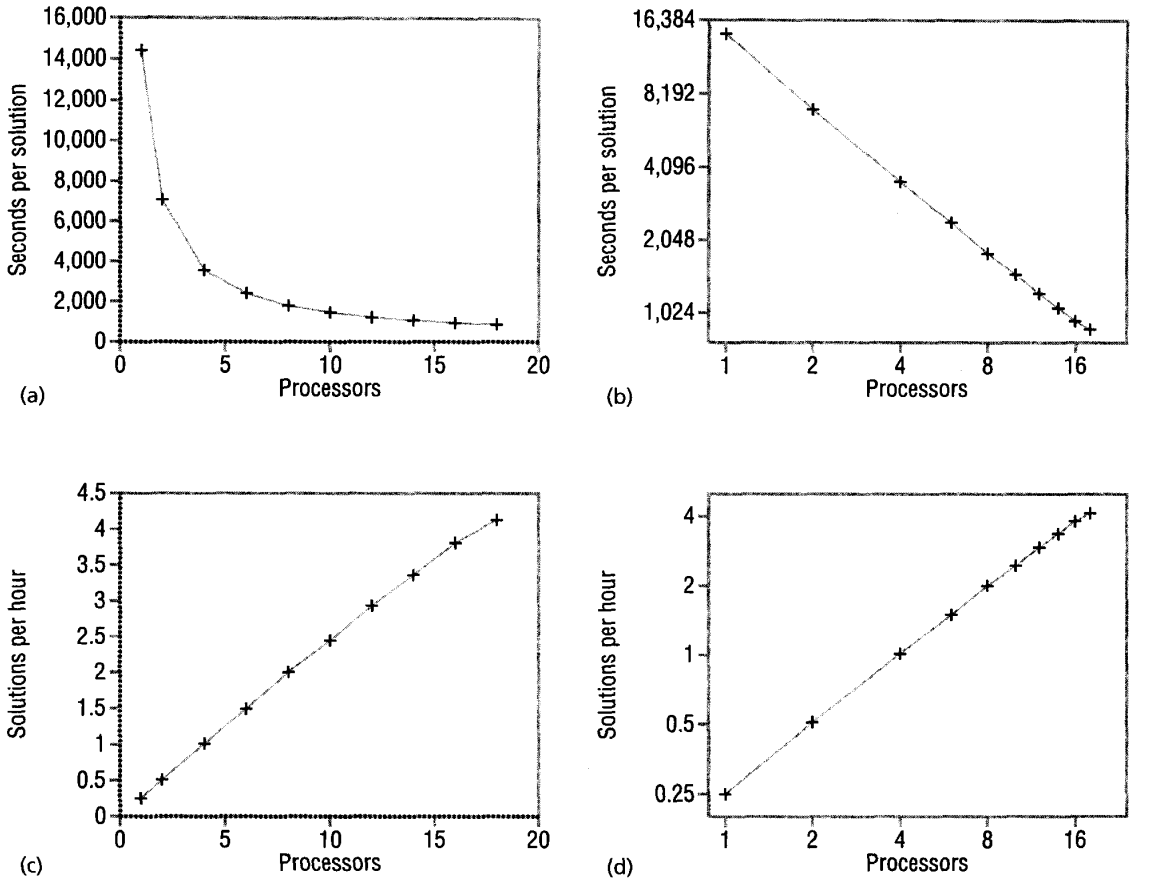


Figure 5. Showing near-linear performance: (a) linear time, (b) log-log time, (c) linear speed, (d) log-log speed.[1]

Linear performance should be linear on paper

For linear time plots, linear performance looks like the function $1/x$. Unfortunately, people have problems recognizing this function, so they cannot easily distinguish linear performance on linear time plots. For example, although they look nearly the same, Figure 5a is near-linear, while Figure 6a is not.

However, people are good at recognizing straight lines; the near-linear performance in Figure 5 and the nonlinear performance in Figure 6 are easier to see in the linear speed, log-log time, and log-log speed plots.

To help readers evaluate how close performance is to linear, the plot function should make linear performance *look* linear (straight) on paper. Linear time plots do not do this, so you should avoid them.

Absolute performance should be linear on paper

When the absolute number of processors and absolute performance are important to your point, you want the number of processors and the resulting performance to be proportional on paper. For example, each additional processor might change the point's horizontal position on paper by one centimeter. The only plot function that achieves this property for processors and performance is the linear speed plot. Also, the linear speed plot is visually similar to speedup plots, so you should just plot speed instead of speedup.

However, there is a danger with linear speed plots. If the dependent variable's origin is not zero, marginally better programs can look much better. This same problem also appears in bar charts that do not have a zero origin, or in bar charts that have a zero origin but that "tear out" the middle part of the bars. For example, Figure 7a exaggerates the performance improvement for increasing processors; Figure 7b does not. So, use zero-based origins.

Occasionally the variation in performance is small, yet you want to emphasize it. Rather than use a non-zero origin, show variation around the mean execution time. Because some entries will be below the mean, as

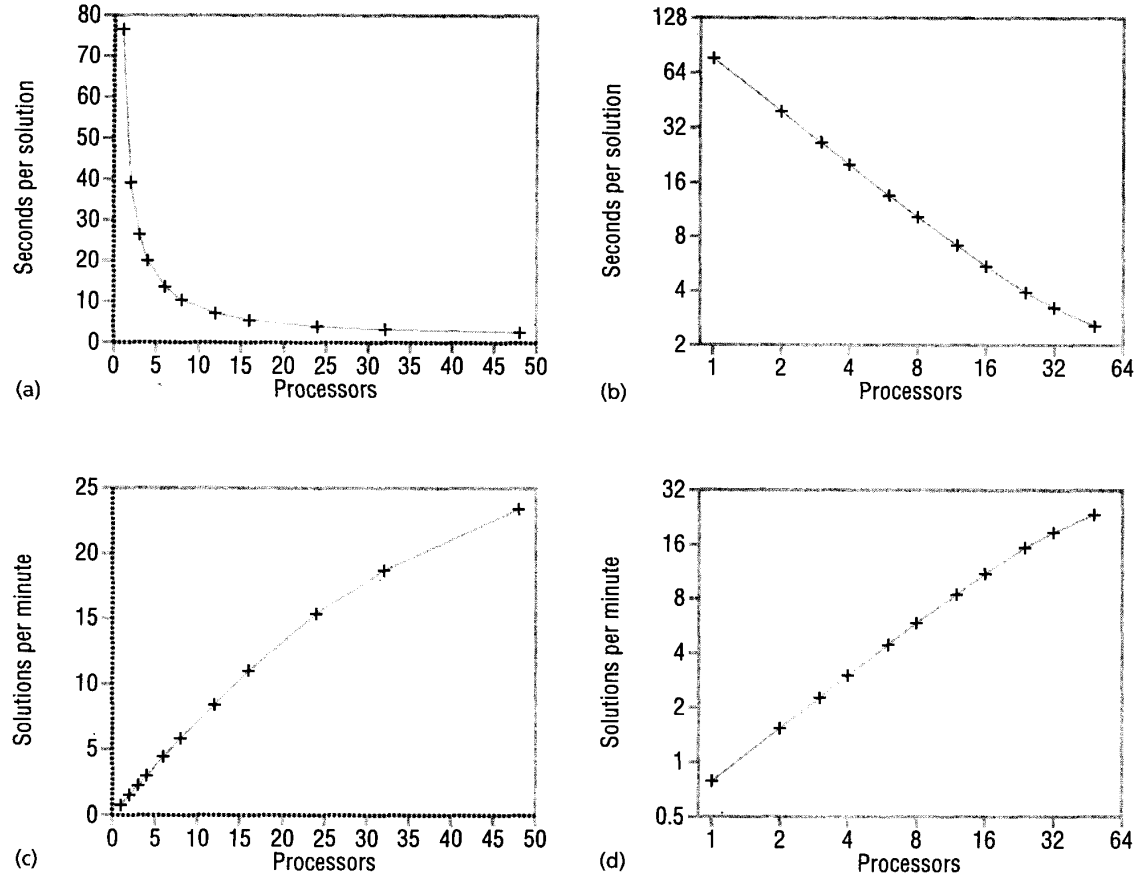


Figure 6. Showing nonlinear performance: (a) linear time, (b) log-log time, (c) linear speed, (d) log-log speed.[7]

in Figure 7c, it will be clearer that the graph is showing variation, and not absolute performance. Figure 7d emphasizes variation even more strongly.

Relative performance should be linear on paper

When relative changes in processors and performance are important to your point, you want them to be proportional on paper. For example, each 10 percent increase in processors might change the paper position by one centimeter. Linear time and linear speed plots do not satisfy this property, but log-log time and log-log speed plots do. Log-log plots give the relative changes equal visual space, so we see the relative performance more easily. For example, Figure 8a shows the speedup of a simulated machine, the Dash multiprocessor, and the ideal (relative to the parallel program on a single processor), in its originally published linear speedup form. In this graph, the simulated machine appears to be significantly (but not excessively) below the ideal performance. Figure 8c shows the same data with a log-log speedup plot. In this graph, we can see that the simulated machine is relatively close to the ideal. In this case, the authors' choice of a linear speedup plot understated the quality of their results.

Because log-log plots show relative performance, they show performance well for both small and large numbers of processors. In contrast, linear speed plots tend not to show relative performance well for small numbers of processors, and linear time plots tend not to show relative performance well for large numbers of processors. For example, the data for low numbers of processors in Figures 8a and 8b is cluttered and hard to distinguish. However, in Figures 8c and 8d, the data for low numbers of processors is much easier to see — so much easier that we can easily identify the artificially introduced anomaly for processor counts 4 and 8 in Figure 8d; we cannot see it easily in Figure 8b.

Performance should be scale-invariant on paper

As discussed earlier, scaling performance results to show

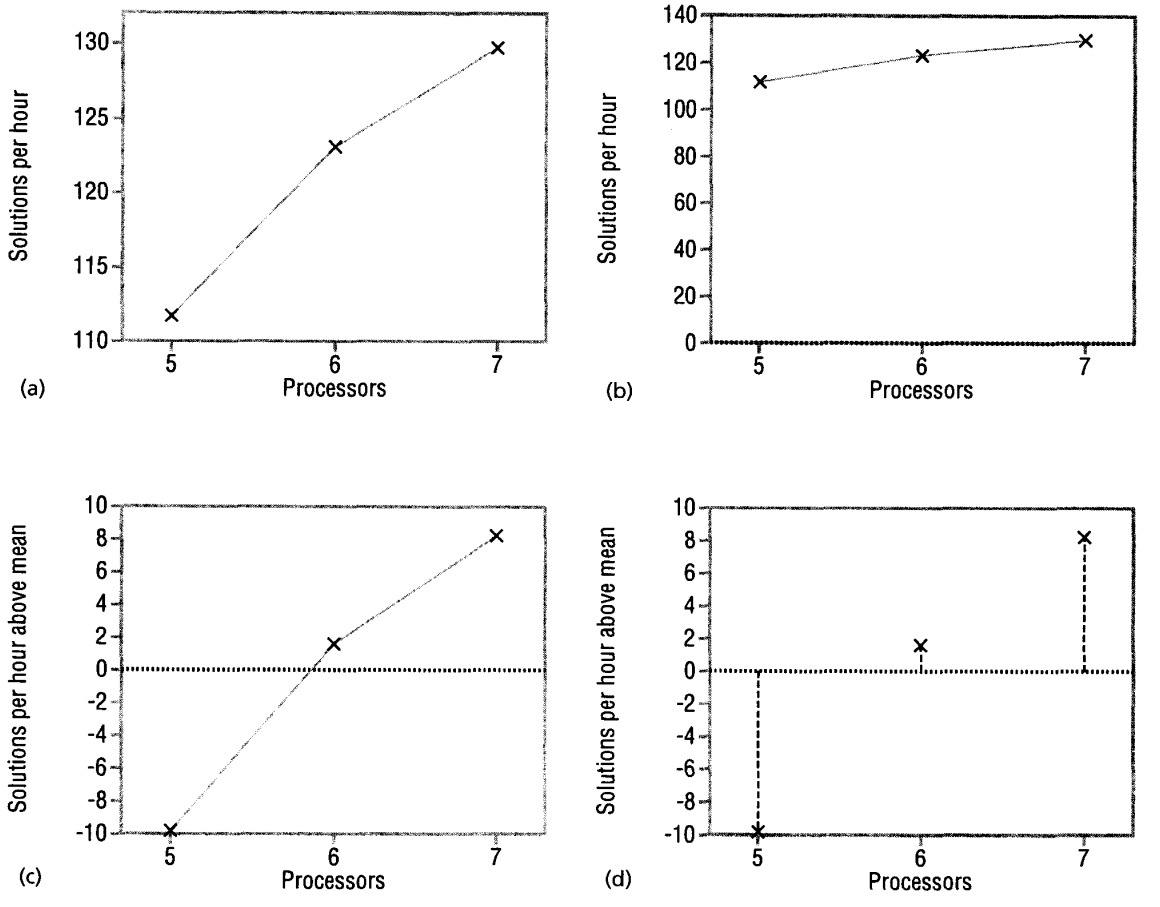


Figure 7. Emphasizing variation in performance: (a) poor origin, (b) good origin, (c) variation around mean, (d) variation around mean emphasized.

speedup may distort results. But we can avoid this problem by taking advantage of a useful property of log-log plot functions—scale invariance. Scale invariance means that doubling each value on the curve does not change the curve's shape. The curve may change position, but it does not change shape. Linear plots do not have this property. For example, in Figure 9a, the Symmetry and Multimax appear to get closer in performance as the number of processors increases, while in Figure 9c they appear to get further apart. However, figures 9b and 9d show a nearly constant distance between the machines at each processor, which shows that program performance differences are relatively constant across all numbers of processors. The fact that the shapes of the curves are independent of processor technology is important when we compare performance on different machines (see "How to compare").

Table 1. Plot properties and functions.

Property	Linear time	Linear speed	Log-log time	Log-log speed
Qualitative changes are obvious		X	X	X
Linear performance is linear on paper		X	X	X
Absolute performance is linear on paper		X		
Relative performance is linear on paper			X	X
Performance is scale-invariant on paper			X	X

Summary of properties

Table 1 summarizes the desirable plot properties and the plot functions that satisfy them. Linear time plots fail to meet any of the desired properties, so you should avoid them. Linear speed plots show absolute performance well, but show relative performance poorly and are not scale-invariant; use them only when absolute performance is important. Log-log plots show absolute performance poorly, but show relative

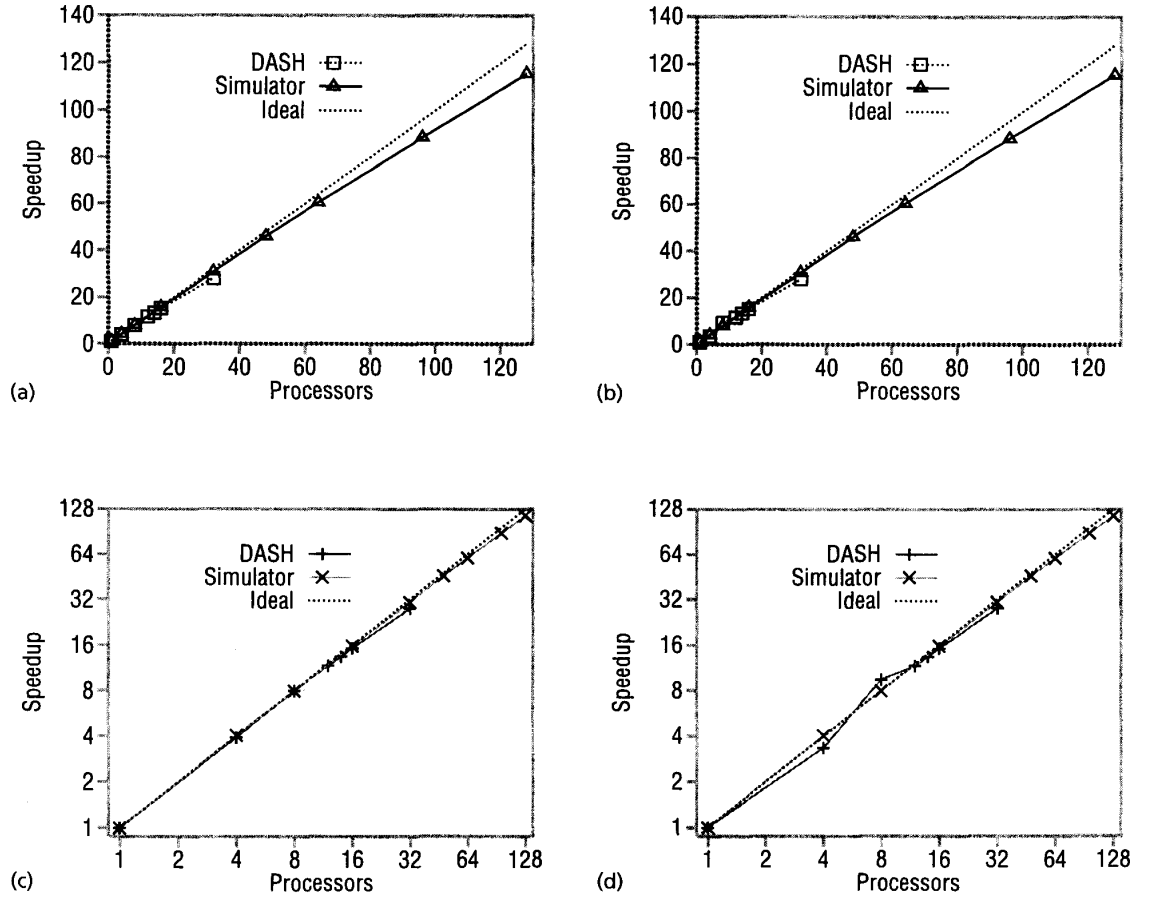


Figure 8. Showing relative performance with log-log plots: (a) linear speedup of original, (b) linear speedup of anomaly, (c) log-log speedup of original, (d) log-log speedup of anomaly.[5]

performance well and are scale-invariant; use them when absolute performance is not important.

So far, log-log time and log-log speed plots share the same properties. Log-log speed plots and linear speed plots show improving performance as a positive slope, but the (usually) opposite slopes of log-log time plots and linear speed plots make them easier to distinguish. I suggest using log-log time plots in preference to log-log speed plots.

Measure points

Nearly all the variables we can control in measuring parallel performance are discrete; they do not vary continuously. However, graphing parallel performance with lines visually implies that performance varies continuously with the number of processors. We all know this is not true, but we often interpret a line to at least mean that you have collected data at every processor. To avoid implying that you have collected more data than you actually have, you should not use lines in your graph, but instead present your measure points. For example, the line-based Figure 10a implies a smooth improvement in performance, while Figure 10b shows that there are no data for many processor counts. The line-based graph leads us to assume the missing data in Figure 10c, but in fact the missing data are as shown in Figure 10d. Plotting your measure points does not distort the amount of data you have collected. Of course, readers will now notice when you have sparse data, so measure at as many processor counts as you can.

Use simple glyphs that clearly show the measure points. In Figure 11 the measure points + and × are easier to see and let readers make finer comparisons than □ and △.

The above advice works fine when the graph contains a single data set or a few widely separated data sets. However, if there are many, closely spaced, or overlapping data sets on a single graph, the measure points alone are hard to interpret. In this case, you should place lines between the measure points of each data set (you must still mark each measure point). These lines lead the reader's eye to "connect the dots," group elements

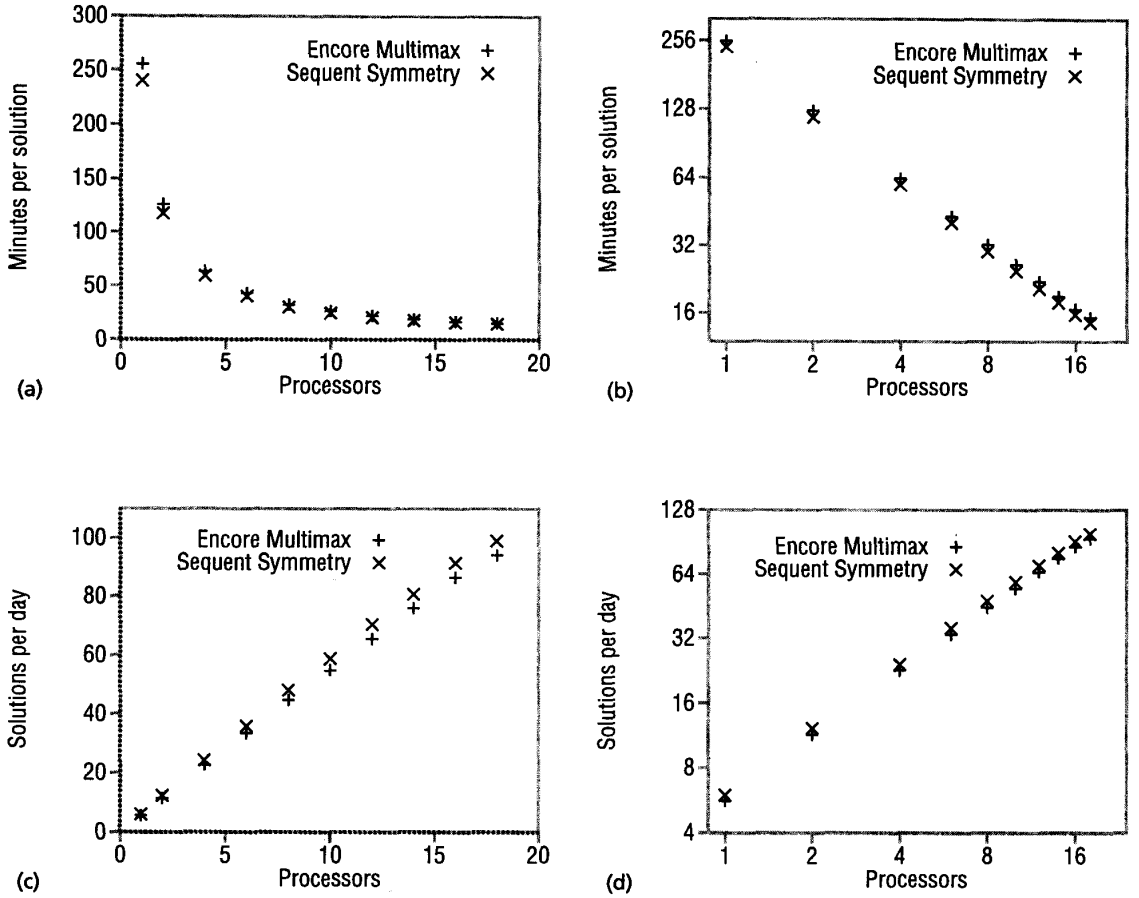


Figure 9. Showing relative performance with scale invariance: (a) linear time, (b) log-log time, (c) linear speed, (d) log-log speed.[1]

into whole data sets, and perceive the *gestalt* of a data set. Note the difference between Figures 12a and 12b.

Distinct measure points alone may not separate data sets well. Sometimes you must also use distinct lines, either with patterns, as in Figure 12b, or with colors. When using colors, keep in mind that some readers might not discern colors well; for example, some people cannot distinguish between red and green. Distinguishing among many different line patterns or colors is difficult, so avoid placing many data sets on a single graph unless they are clearly separated.

When connecting discrete data points with lines, be careful to avoid encouraging the reader to view the distance between *lines* rather than the distance between *points*. For example, in Figure 13a, the heavy lines connecting the data points encourage people to measure the difference in performance by the perpendicular distance between the two lines. As a result, at lower numbers of processors, the two data sets appear much closer than they really are. In contrast, Figure 13b uses a muted line to connect the bold data points, which leads people to measure the distance between the data points. In this case, the connecting lines actually add no information to the graph and should be omitted, as in Figure 2. In contrast, the connecting lines of Figure 6 help highlight the trends, and should be included. Muted lines connect data and highlight trend lines well, without overwhelming the data points and misleading the eye.

Heavy graph border lines also encourage the reader's eye to measure the distance between the graph lines and the border. For log-log plots, the distance between graph lines and border lines is meaningless. You can avoid a meaningless comparison by muting the border lines, as I have throughout the article.

Error bars and multiple data points

I argued earlier that you need to measure multiple execution times to ensure that you have good data. In some

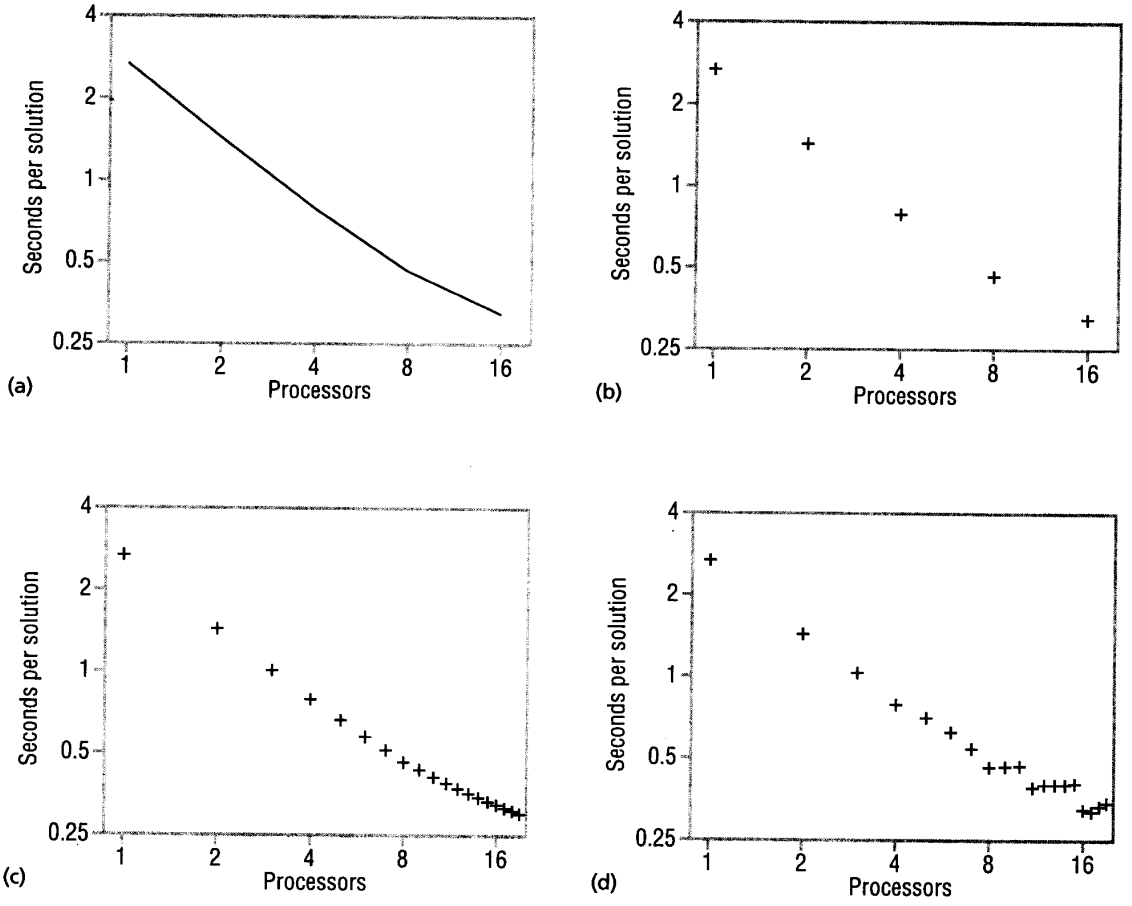


Figure 10. Showing measure points: (a) lines, (b) few points, (c) implied times, (d) actual data.

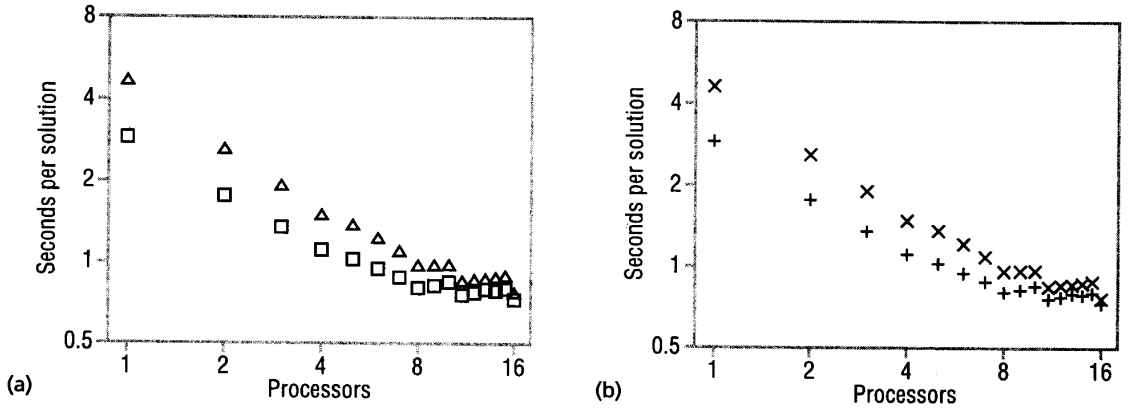


Figure 11. Glyphs without (a) and with (b) clear measure points.

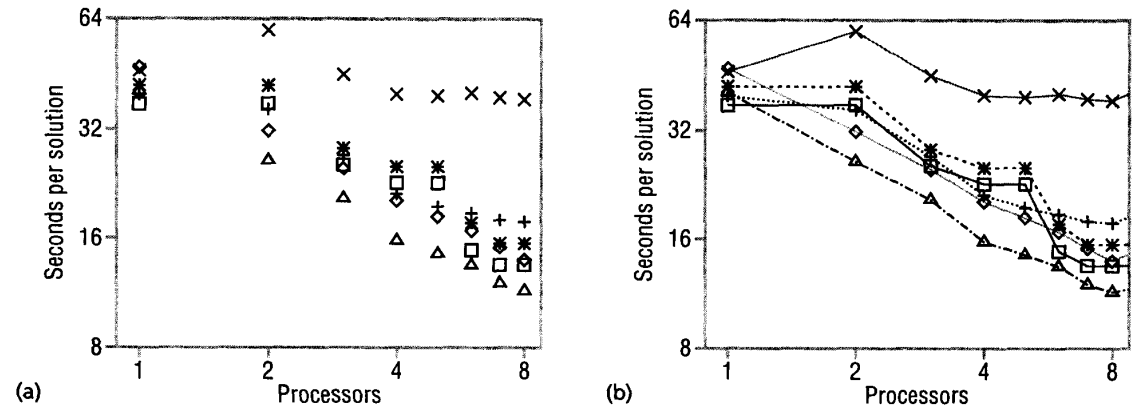


Figure 12. Unconnected (a) and connected (b) measure points for multiple data sets.

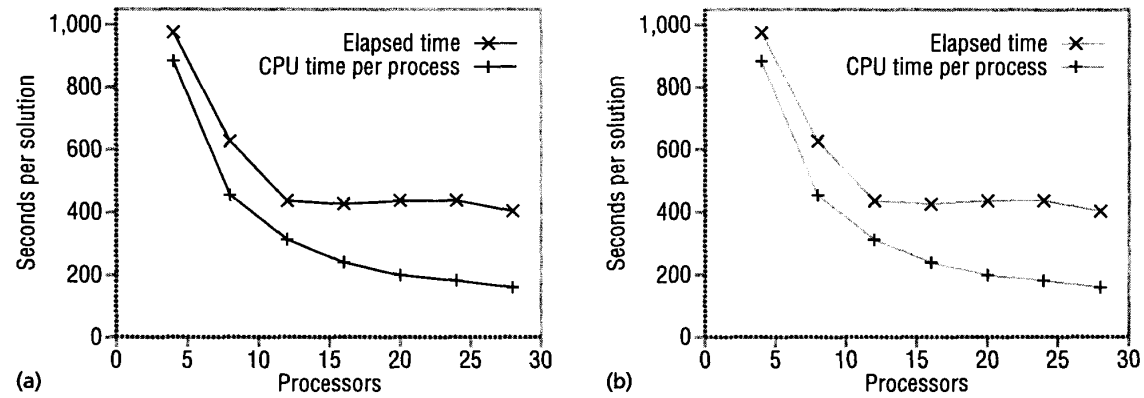


Figure 13. Bold (a) and muted (b) connecting lines.[1]

cases, the variance (or standard deviation) in your data will be so small that you can almost ignore it in your presentation. Almost. You should state in the text that the data has an insignificant variance, so your reader is aware of your data's consistency.

When your data varies significantly and is normally distributed around the mean, simply put error bars on your measure points. An error bar shows the mean value and a vertical bar indicating the mean plus or minus one standard deviation (see Figures 14a and 15a). Error bars concisely represent the variability in normal distributions, and let readers visually perceive the variance relative to the data values.

When your measure points do not have a normal distribution, you should plot all the measured data. Figures 14a and 15a show variability via error bars, but provide no clue as to the reason for it; however, Figures 14b and 15b suggest the reason. In Figure 14b (eight points per processor), for few processors most points are consistent, but with some outliers. As we start to use most of the machine's processors, points become very inconsistent. Knowing that the machine is time-shared, we can infer that contention with other users is upsetting the data. In Figure 15b there are 16 points per processor, but at most three are visible because the rest have exactly the same values. This exactness can be explained by clock granularity, which indicates that there could be timing problems.

If your data has significant observed variability, you should explain it in the text and present it on the graph, either through error bars or through plotting all data points.

What to compare

The most useful way to present a program's performance is to compare it with other programs. To make a comparison, you must change one or more of the many independent variables that go into parallel performance: the algorithm (as defined by the dataflow), the parallelization of the algorithm, the programming language, the compiler, the automatic optimization effort, the manual optimization effort, the machine, the numeric precision, the memory available, the number of processors, the problem to be solved, the input data, and so on. To make reasonable conclusions from the comparison, your reader must know *all* the independent variables you changed, not just the primary variable. For example, if I tell you that a Mandelbrot program executes faster on one machine than another, you can (and probably will) conclude that the first machine is faster for this kind of application. But if I then tell you that the program is optimized for the first machine but not for the second, you can no longer make the same conclusion.

With so many independent variables in parallel performance, it is easy to dilute results or distort performance by changing too many variables at once. It is best to change as few as possible, which makes your results stronger. Some changes may be unavoidable, such as using two different compilers to execute a program on two different machines. To avoid misleading your reader, you should clearly document those variables that change but are not part of the comparison.

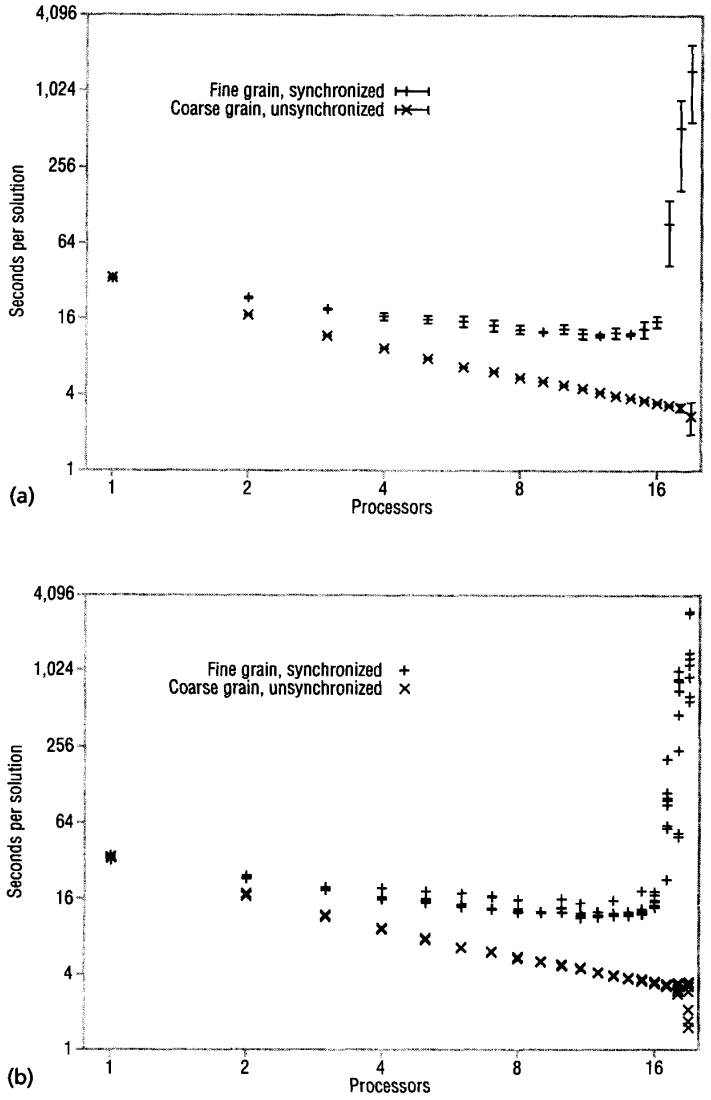


Figure 14. Showing statistical measures with contention: (a) error bars showing standard deviation; (b) showing all measure points.

David Bailey proposes nine guidelines for avoiding the distortions that arise from undocumented changes in independent variables:[8]

- For a well-known benchmark, compare results only for the same version of the benchmark.
- Do not compare projections or extrapolations with actual performance rates.
- Compare results only with similar efforts when tuning performance.
- Compare run times, preferably using wall clock time, not Mflops and the like.
- Only present Mflops rates when based on consistent flop counts.
- Base speedup on a reasonably tuned sequential program. Clearly identify scaled speedup.
- Disclose ancillary information that would significantly affect the interpretation of results.
- Include any important qualifying information in abstracts, figures, and tables.
- Include all the information necessary for other scientists to accurately reproduce the performance results presented in the paper.

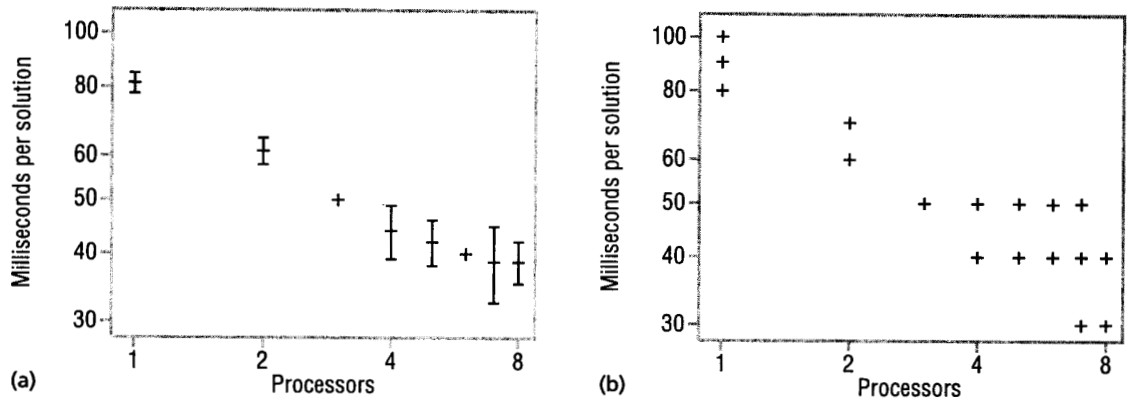


Figure 15. Showing statistical measures with granularity: (a) error bars showing standard deviation; (b) showing all measure points.

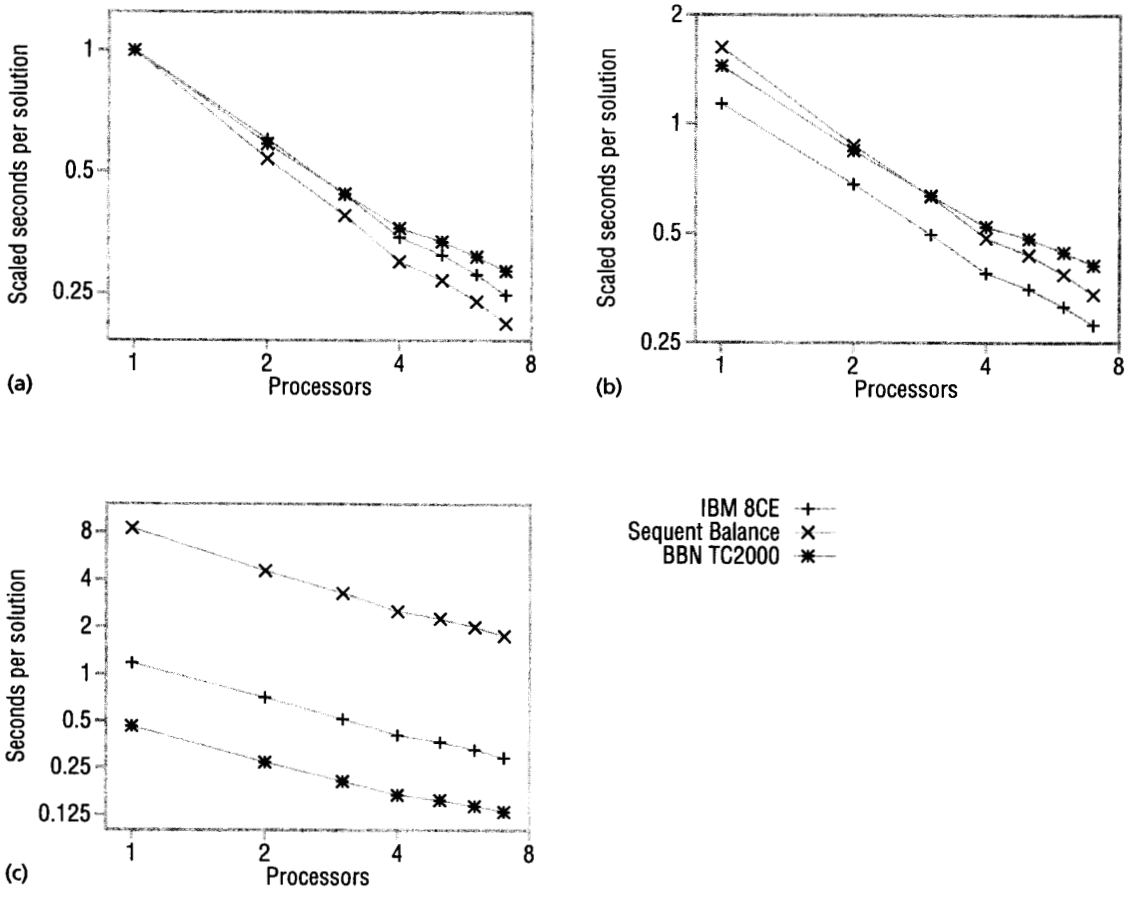


Figure 16. The effect of scaling on performance graphs: (a) time scaled to a single processor, (b) time scaled to sequential time, (c) elapsed time.

R.H.F. Jackson and his colleagues provide more comprehensive guidelines.[9]

How to compare

How you compare performance on the printed page depends on the point you want to make. Are you comparing machines, programming systems (language and compiler), algorithms, parallelizations of algorithms, or different problem sizes? Do you want to compare observed performance with "perfect" performance?

Speed of machines, programming systems, or algorithms

If you are comparing machines, programming systems, or algorithms, elapsed time is what matters, so you should present that. All the reasons that make speedup a poor way to present the performance of one parallel program also make it a poor way to compare two programs. Comparable speedup depends on a consistent base, which is difficult to achieve because bases may differ in several independent variables. Comparing speedup also fails to account for algorithms or processors that are unnecessarily slow when run sequentially. For example, the BBN Butterfly I had no floating-point hardware, so programs using a significant amount of floating-point computation spent so much time implementing floating-point operations that any reasonable communication took a very small fraction of the total time; such programs attained linear speedup quite easily. Comparing such a program's speedup on the Butterfly I with its speedup on a machine with floating-point hardware is a gross distortion.

Presenting unadorned execution time also keeps changing technology in perspective—in multiprocessors, processor speed has improved faster than communication speed,[10] which means that newer machines show poorer speedup even though they execute programs faster. For example, our conclusions about which machine in Figure 16 is better depend on whether we look at time scaled to the time of the single-processor parallel program (Figure 16a), time scaled to the time of the sequential program (Figure 16b), or elapsed time (Figure 16c). To avoid distortion, compare machines, programming systems, and algorithms by elapsed time.

Writers often compare speedup to compensate for differences in single-processor performance. However, the scale invariance of log-log plots lets us make such comparisons without computing speedup. If you compare elapsed time on log-log plots, the shapes of the curves are independent of processor technology, and you can see scaling trends for one program/machine independent of the others. For example, in Figure 17 we can see that the TC2000 is always faster than the Symmetry for small numbers of processors, but also that the TC2000 does not improve, so the Symmetry may well perform better at larger numbers of processors. We can also see that the 8CE improves faster than the Symmetry, because the slope for the 8CE is greater. Unless comparisons based on absolute performance are important, use log-log plots to compare execution times.

How machines affect algorithms and parallelizations

If the point of your comparison is not speed, but rather how machines influence algorithms or parallelizations,

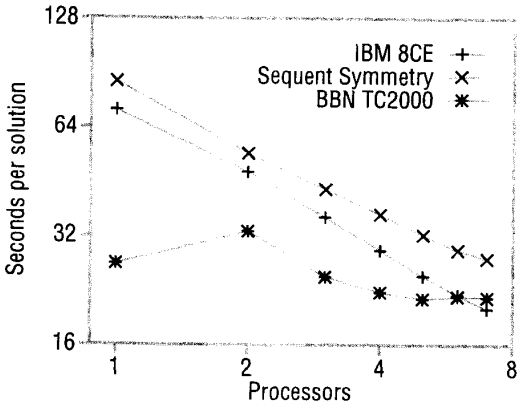


Figure 17. Elapsed time and scale invariance.

Other resources

I recommend the books of Edward Tufte to any author presenting quantitative information. *The Visual Display of Quantitative Information* (1983) and *Envisioning Information* (1990) are available from Graphics Press, Box 430, Cheshire, CT 06410. Many graphical plotting packages do not follow Tufte's recommendations, in which case you will have to balance the quality of the presentation with your effort, including possibly modifying the plotting package.

Rai Jain also discusses these issues (in the context of sequential performance analysis) in *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*, published in 1991 by John Wiley & Sons. Jain also offers more detailed advice on measuring and analyzing performance than there is room for in a single magazine article.

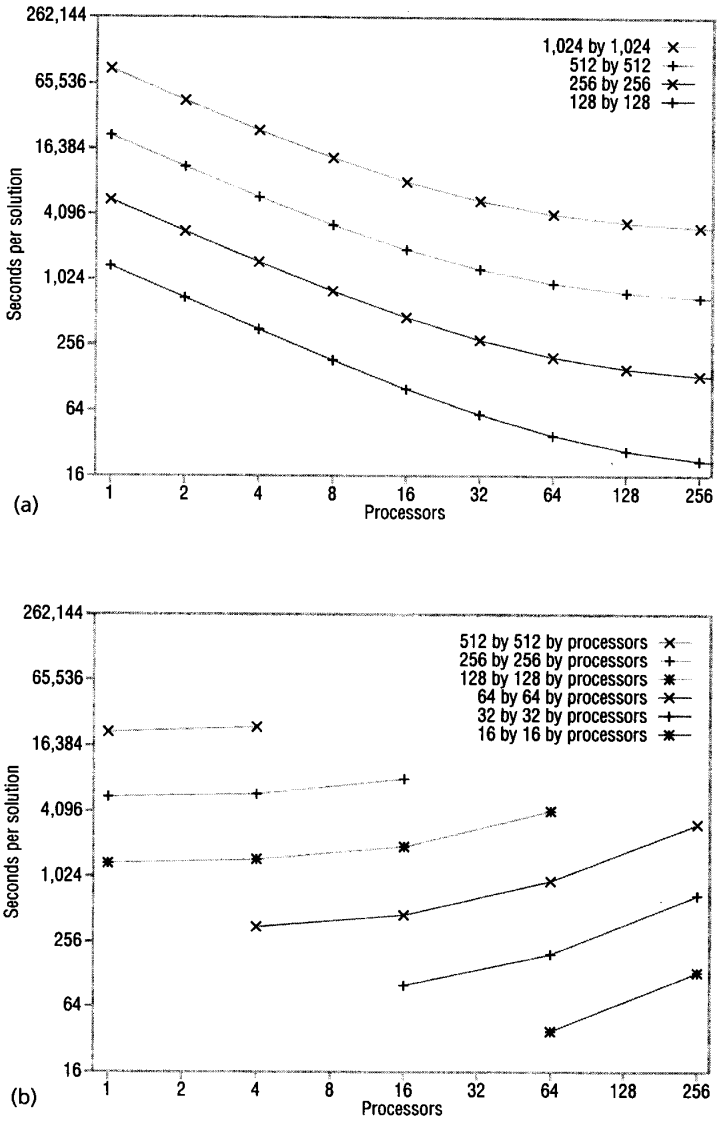


Figure 18. Fixed (a) and scaled (b) problem size.[11]

then scaling elapsed time *is* appropriate. However, you must be careful to scale to the execution time of a sequential program, not the parallel program on a single processor. The overhead for creating and managing parallelism varies widely, even for a single processor, which could distort the true performance (see Figures 16a and 16b). In this case, Figure 16b is the best representation because it explicitly presents parallelization overhead. So, if you are comparing the behavior of algorithms or parallelizations on different machines, use log-log time scaled to the sequential processor case.

Problem sizes

If you are comparing different problem sizes, the straightforward approach is to plot time versus processors for each relevant problem size. Log-log plots are particularly useful here because they show the relationship between relative increases in problem size and execution time. For example, in Figure 18a we see that quadrupling the problem size on a single processor quadruples the execution time, and that four processors can complete the larger problem in almost the same time as a single processor can complete the smaller problem.

This relationship is clearer in Figure 18b, where the lines connect a quadrupled problem size with a quadrupled number of processors—in other words, the lines show problem size scaled with the number of processors. If the performance improvement were perfect, we would see horizontal lines. Where the lines are not horizontal, a scaling problem exists.

Perfect performance

Sometimes you might want to compare observed performance with "perfect" performance. This usually involves scaling a sequential program, but because the base sequential program is usually subject to controversy, a better approach than scaling the actual data is to put the sequential program's extrapolated performance on the graph as a separate line. That is, you should show both the parallel program's actual performance and the sequential program's perfect speedup. For example, Figure 19a shows the measured parallel performance with data points, and shows the times of the "perfectly parallel" sequential program with a line.

Sometimes a program parallelizes so poorly as to require extending the unextrapolated sequential program time across the graph, as in Figure 19b. This example shows that near 22 processors, the parallel program takes as much time as the sequential program.

If you are studying different parallel algorithms, you

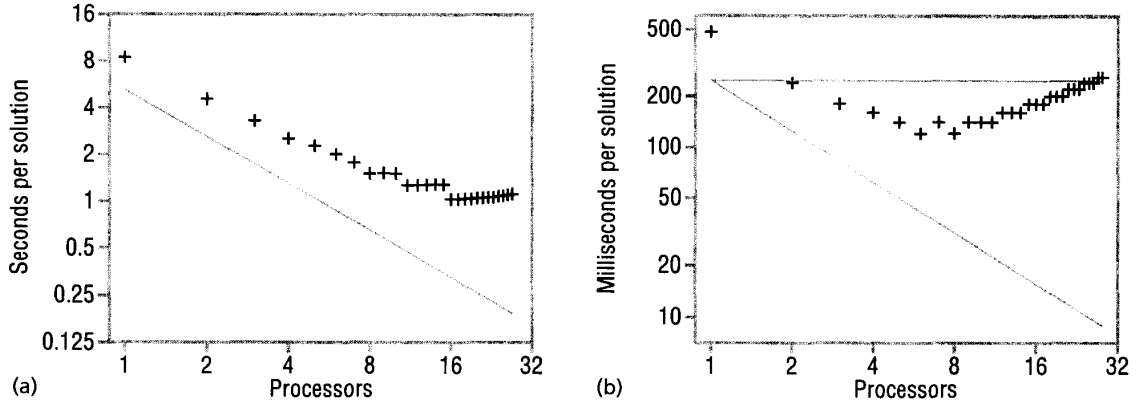


Figure 19. Showing "perfect" performance: (a) sequential extrapolated, (b) carried and extrapolated.

should use the time of the best-known sequential program for your extrapolation. If you are studying different parallelizations of a given algorithm, you should use the time of that algorithm's sequential version. In either case, putting the extrapolated "perfect" performance on paper separately from the actual data helps your reader understand the interpretation implicit in perfect performance.

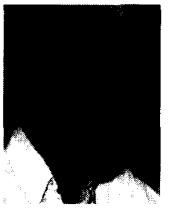
Effectively presenting parallel performance will make the time you spend investigating it more effective as well. These guidelines should help you present parallel performance with less distortion and better use of space and time. On occasion, you may better meet this goal by violating one or more of these guidelines, but making explicit your reasons for doing so should lead you to more effective presentations.

Acknowledgments

The comments of Margaret Burnett, Jesse Chaney, Mark Clement, Phyllis Crandal, Kevin Djang, Cherri Pancake, Rajeev Pandey, and the anonymous reviewers were very helpful. Jaswinder Singh provided some of the source data.

References

1. A. Deshpande and M. Schultz, "Efficient Parallel Programming with Linda," *Proc. Supercomputing '92*, IEEE Computer Society Press, Los Alamitos, Calif., 1992, pp. 238–244.
2. R.S. Barr and B.L. Hickman, "On Reporting the Speedup of Parallel Algorithms: A Survey of Issues and Experts," Tech. Report No. 91-CSE-20, Dept. of Computer Science and Eng., Southern Methodist Univ., Dallas, Tex., 1991.
3. J.L. Gustafson, G.R. Montry, and R.E. Benner, "Development of Parallel Methods for a 1,024-Processor Hypercube," *SIAM J. Scientific and Statistical Computing*, Vol. 9, No. 4, July 1988, pp. 609–638.
4. X.-H. Sun and L. Ni, "Scalable Problems and Memory-Bound Speedup," *J. Parallel and Distributed Computing*, Vol. 19, No. 1, Sept. 1993, pp. 27–37.
5. J.P. Singh, J.L. Hennessy, and A. Gupta, "Scaling Parallel Programs for Multiprocessors: Methodology and Examples," *Computer*, Vol. 26, No. 7, July 1993, pp. 42–50.
6. V. Faber, O.M. Lubeck, and A.B. White, Jr., "Comments on the Paper 'Parallel Efficiency Can Be Greater than Unity,'" *Parallel Computing*, Vol. 4, No. 2, Apr. 1987, pp. 209–210.
7. L. Crowl, *Architectural Adaptability in Parallel Programming*, doctoral dissertation, Tech. Report 381, Computer Science Dept., Univ. of Rochester, N.Y., 1991.
8. D.H. Bailey, "Misleading Performance in the Supercomputing Field," *Proc. Supercomputing '92*, IEEE Computer Society Press, Los Alamitos, Calif., 1992, pp. 155–158.
9. R.H.F. Jackson et al., "Guidelines for Reporting Results of Computational Experiments: Report of the Ad Hoc Committee," *Mathematical Programming*, Vol. 49, No. 3, Series A, Jan. 1991, pp. 413–426.
10. M.L. Barton and G.R. Withers, "Computer Performance as a Function of the Speed, Quantity, and Cost of the Processors," *Proc. Supercomputing '89*, IEEE Computer Society Press, Los Alamitos, Calif., 1989, pp. 759–764.
11. S. Seetharamakrishnan, "Strengths and Weaknesses of Data Parallel C," master's thesis, Oregon State Univ., 1992.



Lawrence Crowl is an assistant professor of computer science at Oregon State University. His broad research area is parallel systems, including programming techniques, programming languages, operating systems, and their architectural support. He received his PhD in computer science from the University of Rochester in 1991. He is a member of the ACM and the Sigma Xi Research Society. He can be reached at the Computer Science Dept., Oregon State Univ., Corvallis, OR 97331-3202; Internet, crowl@cs.orst.edu